

Directory

os module – This module is used to perform simple operation on directories. This module represents operating system dependent functionality.

```
import os
```

```
import os

# Get current working directory

print("Current Directory:", os.getcwd())

# List all files and folders in current directory

print("Files & Folders:", os.listdir())

# Create a new directory

os.mkdir("test_folder")

print("Created folder: test_folder")

# Rename a directory

os.rename("test_folder", "renamed_folder")

print("Renamed folder")

# Remove a directory

os.rmdir("renamed_folder")

print("Removed folder")

# Check if a file/folder exists

print("Does file exist?", os.path.exists("data.csv"))

# Join paths (safe for Windows/Linux)

path = os.path.join(os.getcwd(), "example.txt")

print("Full path:", path)
```

Exception

An exception is a runtime error which can be handled by the programmer.

All exceptions are represented as classes in Python.

Type of Exception:-

- Built-in Exception – Exceptions which are already available in Python Language. The base class for all built-in exceptions is BaseException class.
- User Defined Exception – A programmer can create his own exceptions, called user-defined exceptions.

Need of Exception Handling

- When an exception occurs, the program terminates suddenly.
- Suddenly termination of program may corrupt the program.
- Exception may cause data loss from the database or a file.

```
try:

    x = 5 / 1

except ZeroDivisionError:

    print("✗ Cannot divide by zero")

else:

    print("✓ Division successful:", x)

finally:

    print("🏁 Done")
```

```
try:
    print(10 / 0)
except Exception as e:
    print("⚠ Error:", e)
```

try → risky code

except → handles error

else → runs if no error

finally → always runs

User Defined Exception

A programmer can create his own exceptions, called user-defined exceptions or Custom Exception.

- Creating Exception Class using Exception Class as a Base Class
- Raising Exception
- Handling Exception

Creating Exception

We can create our own exception by creating a sub class to built-in Exception class.

```
class MyException(Exception):  
    pass
```

```
class MyException(Exception):  
    def __init__(self, arg):  
        self.msg = arg
```

Handling Exception

Using try and except block Programmer can handle exceptions.

```
try:
```

```
    statement
```

```
except MyException as mye:
```

```
    statement
```


Logging

Logging is a way to **track events** that happen while your program runs.

Instead of using `print()`, you use the `logging` module, which gives you more **control, levels, and formatting**.

Useful for debugging, auditing, error tracking, a

Basic Example

```
import logging

# Configure logging
logging.basicConfig(level=logging.INFO)

logging.debug("This is a debug message")
logging.info("This is an info message")
logging.warning("This is a warning")
logging.error("This is an error")
logging.critical("This is critical")
```

Level	Use Case
DEBUG	Detailed info (development, troubleshooting)
INFO	General info about program execution
WARNING	Something unexpected but not breaking
ERROR	A serious problem occurred
CRITICAL	Very serious error, program may not continue

```
import logging
```

```
logging.basicConfig(  
    filename="app.log",      # log file name  
    level=logging.DEBUG,    # capture all logs  
    format="%(asctime)s - %(levelname)s - %(message)s"  
)
```

```
logging.info("Application started")
```

```
logging.warning("Low disk space")
```

```
logging.error("File not found")
```

What is Polymorphism?

- **Polymorphism** = *“one name, many forms”*.
- It allows the same function/method/operator to **behave differently based on the object or input type**.
- In Python, polymorphism is **not strictly enforced** (like Java/C++), because Python is **dynamically typed**.

1. Polymorphism with Functions/Methods

Different classes can have methods with the **same name**, but their behavior depends on the object.

```
class Dog:
```

```
    def sound(self):
```

```
        return "Bark"
```

```
class Cat:
```

```
    def sound(self):
```

```
        return "Meow"
```

```
# Polymorphism: same method name, different behavior
```

```
for animal in [Dog(), Cat()]:
```

```
    print(animal.sound())
```

Polymorphism with Inheritance (Method Overriding)

Child class **overrides** parent class method.

```
class Animal:
```

```
    def move(self):
```

```
        print("Animals can move")
```

```
class Dog(Animal):
```

```
    def move(self):
```

```
        print("Dogs can run and walk")
```

```
obj1 = Animal()
```

```
obj2 = Dog()
```

```
obj1.move() # Animals can move
```

```
obj2.move() # Dogs can run and walk
```

Polymorphism with Functions (Duck Typing)

Python uses **duck typing**:

"If it looks like a duck and quacks like a duck, it's a duck."

You don't care about the object's type, just whether it implements the required method.

```
class Duck:
```

```
    def swim(self):
```

```
        print("Duck swims in the pond")
```

```
class Fish:
```

```
    def swim(self):
```

```
        print("Fish swims in the river")
```

```
def make_it_swim(obj):
```

```
    obj.swim()
```

```
make_it_swim(Duck()) # Duck swims in the pond
```

```
make_it_swim(Fish()) # Fish swims in the river
```

Operator Overloading (Built-in Polymorphism)

Python lets you **redefine operators** for custom classes.

```
class Book:
```

```
    def __init__(self, pages):
```

```
        self.pages = pages
```

```
    def __gt__(self, other): # Greater than dunder method/magic method
```

```
        return self.pages > other.pages
```

```
b1 = Book(120)
```

```
b2 = Book(200)
```

```
print(b1 > b2) # False
```

```
print(b2 > b1) # True
```


What is `super()` method?

- `super()` is used to **call methods from a parent (super) class** inside a child class.
- Commonly used in **inheritance** to extend or reuse parent methods without rewriting them.
- Avoids duplicating parent code.

```
class A:
```

```
    def display(self):
```

```
        print("Class A")
```

```
class B(A):
```

```
    def display(self):
```

```
        print("Class B")
```

```
        super().display()
```

```
class C(B):
```

```
    def display(self):
```

```
        print("Class C")
```

```
        super().display()
```

```
c = C()
```

```
c.display()
```

What is a Decorator?

- A **decorator** is a special function that **adds extra functionality to another function** without modifying its source code.
- It uses the **@decorator_name** syntax.
- Decorators are heavily used in frameworks like **Flask, Django, FastAPI**.

```
def my_decorator(func):  
    def wrapper():  
        print("Before function call")  
        func()  
        print("After function call")  
    return wrapper
```

```
@my_decorator  
def say_hello():  
    print("Hello, World!")
```

```
say_hello()
```

argparse

It is used to **handle command-line arguments** for Python scripts (like `python script.py --name John --age 25`)

How to Run in terminal/command prompt:

```
python example.py --name Alice --age 30
```

argparse

```
import argparse
```

```
# Step 1: Create parser
```

```
parser = argparse.ArgumentParser(description="Simple Argument Parser Example")
```

```
# Step 2: Add arguments
```

```
parser.add_argument("--name", type=str, help="Enter your name")
```

```
parser.add_argument("--age", type=int, help="Enter your age")
```

```
# Step 3: Parse arguments
```

```
args = parser.parse_args()
```

```
# Step 4: Use arguments
```

```
print(f"Hello, {args.name}! You are {args.age} years old.")
```


